

Verborum — Android Development Documentation

Repository: github.com/ColdTea-Projects/verborum_android · **Target location of this file:** [docs/android-development.md](#) **Related documents:** *Verborum Frontend-Backend Integration* (backend repo, [docs/integration/frontend-backend-integration.md](#)), backend agent docs ([verborum_ms/docs/agent/](#))

1. Project Goals

Verborum is a personal vocabulary platform: users build their own dictionaries of word pairs between two languages, enrich each word with real grammatical detail (gender, plural, verb forms, aspect, readings), practice them, and — in later versions — share dictionaries on a marketplace and receive community-powered suggestions.

The Android app is the **first and reference client**. Its goals:

1. Make creating a *linguistically rich* word entry fast — not just "word = translation", but the forms a learner actually needs, tailored per language.
2. Work reliably as a local-first app: the device database is the working copy, the backend is the durable store.
3. Ship together with the backend: **nothing is released until Android + Backend are ready as a pair**. At release the app will be online and require a login; the current guest/offline mode is a development state.

2. Current Features & Abilities

Area	Status	Notes
Onboarding tour		Welcome flow on first launch
Dictionary CRUD		Create/edit/delete, language pair, name
Word creation		Multi-meaning entries, per-language grammar forms, gender/article chips
Word editing		Stored entries parse back into the create form
Practice — Self practice		Flip-style self-check
Practice — Multiple choice		Generated distractors
Sync engine		Offline-first, upload-then-download (see §5)
Marketplace ("Forum" tab)	stub	"Coming soon" screen; waits for backend <code>ms_marketplace</code>
Authentication	none	Runs entirely as guest user (see §7, §8.2)
Word suggestions (Autofil)		Backend V2 feature

Dictionary languages currently selectable (10): en, de, fr, es, it, pt, nl, lt, tr, az.

3. Architecture

3.1 Modules

```
verborum_android/  
├─ app/           # Navigation shell only (NavigationCentral, bottom bar, Screens)  
├─ bibliotheca/   # The dictionary feature: everything vocabulary-related  
├─ forum/         # Marketplace feature (stub)  
├─ core/          # Shared: theme, BaseViewModel, network plumbing, base URL  
└─ buildSrc/      # Version/config management (Configuration.kt, version.properties)
```

3.2 Stack

Kotlin 2.1 · Jetpack Compose + Material 3 · Hilt · Room · Retrofit + kotlinx.serialization · MockK/JUnit for tests · minSdk 23, compileSdk 35, Java 17.

3.3 Layering (per feature slice)

Each domain concept (`dictionary/` , `word/`) is a vertical slice with three model tiers and conversion functions living next to the models:

```
data/ — db/ (Room: XEntity, XDao)  api/ (Retrofit: XApi, request/response models)  
domain/ — model/ (X)  usecases/local/  usecases/api/  
ui/ ——— model/ (XUi)  screens & composables (ViewModels talk ONLY to Services)
```

`Service` classes (`DictionaryService` , `WordService` , `SyncService` , `UploadService` , `OnboardingService`) orchestrate use cases and are the single API the ViewModels see. Use cases are single-verb classes. `XEntity ↔ X ↔ XUi` conversions keep Room, domain, and Compose models separate.

The repo also ships its own Claude Code setup (`.claude/`): agents `android-build` , `android-data` , `android-test` , `android-ui` and skills `android-dev` , `git-workflow` , `gradle-toolchain` , `scaffold-feature` , `write-tests` .

4. Word Storage — the Canonical Data Contract

This section defines how a word is stored. It is **the** contract: the backend stores these values opaquely (its `words.word` , `words.word_meta` , `words.translation_meta` columns are `json` typed), and every future client (web, iOS) must produce and parse the same shapes. The implementation lives in `bibliotheca/.../word/ui/createword/model/` (`WordFormRules.kt` , `LanguageGrammar.kt` , `FieldKey.kt` , `Gender.kt` , `WordType.kt`) and `word/ui/model/WordMeta.kt` .

4.1 The word column — surfaces as a JSON array

The stored word text is a JSON array of per-meaning *surface forms* — one string per meaning, article included where the language composes one:

```
["der Apfel"]  
["kaufen", "erwerben"]  
["l'eau"]
```

JSON escaping makes any user text safe (including `/`). Blank meanings are dropped.

4.2 The meta column — one JSON object

```
{
  "lang": "de",
  "type": "verb",
  "genders": ["m", ""],
  "fields": { "past": ["kaufte", "erwarb"], "aux": ["haben", "haben"] }
}
```

Rules:

- **lang** — lowercase two-letter code of this side's language.
- **type** — the part of speech (**noun**, **verb**, **adjective**, **adverb**, **preposition**, **pronoun**, **numeral**, **conjunction**, **interjection**, **article**). **Absent for free text.**
- **genders** — list of gender codes (**m**, **f**, **n**, **c**), **index-aligned** with the surfaces array (one entry per meaning, empty string where a meaning has no gender). Omitted entirely when no meaning has a gender.
- **fields** — map of grammatical form key → list of values, again **index-aligned per meaning**. A key blank in every meaning is omitted. Unknown keys must be ignored by parsers (this is the schema-evolution rule).
- Serialization order of **fields** follows the **FieldKey** enum declaration order.

4.3 Current field keys

metaKey	Meaning	Used by (today)
plural	plural form	gendered languages + en
feminine	feminine adjective form	fr, es, it, pt, lt
comparative / superlative	comparison forms	en, de, nl, fr, es, it, pt, lt
present	3rd-person present	lt
past	past / preterite	en, de, nl
past3	3rd-person past	lt
participle	past participle	en, de, nl, fr, it, es, pt
aux	auxiliary verb (choice chip)	de (haben/sein), nl, fr, it

LanguageGrammar is the single source of truth for which language exposes which fields for which word type, gender options, article composition (**der Apfel**, **l'eau**, **lo studente** — with French/Italian elision), and the inverse **extractBaseWord** used when editing.

5. Sync Engine

Room is the working store; every row carries **isSynced** and an **is_deleted** tombstone.

1. **Upload first** (**UploadPendingChangesUseCase**): everything **isSynced = false** is pushed — deletions as DELETE calls, upserts as POST/PUT. Rows are marked synced only on server success.
2. **Download-merge** (**SyncUserDictionariesUseCase**): server state is fetched; local unsynced changes win; tombstoned rows are never resurrected; a **null** API response means "no information — touch nothing".
3. Deletes are remote-first: local hard-delete only after the server confirms.

The device currently runs as a fixed guest identity (**GUEST_USER_ID**, the all-zeros UUID). Ownership of data at login time is handled by the **guest-data migration rule** — defined in the Integration document §6.4 and summarized in §8.2 below.

6. Backend Endpoints & Requirements (Android view)

Base URL today: `http://192.168.0.241:8085/` (LAN, straight to `ms_dictionary`, cleartext — development only; see §8.3). All request/response DTOs mirror the backend contract (`Response` / `ErrorResponse` envelope for mutations; plain DTO lists for reads).

6.1 In use today (`ms_dictionary`)

Method & path	Used for
<code>POST /dictionaries/</code> · <code>PUT /dictionaries/</code>	Upload created/edited dictionaries
<code>DELETE /dictionaries/{dictionaryId}</code>	Remote-first dictionary delete
<code>GET /dictionaries/{userId}</code>	Download all dictionaries of the user
<code>POST /words</code> · <code>PUT /words</code>	Upload words as bundles grouped by dictionaryId
<code>DELETE /words/{wordId}</code> · <code>DELETE /words/dictionary/{dictionaryId}</code>	Word deletes
<code>GET /words/user/{userId}</code>	Download all words of the user

Also available since backend Phase 0 (not yet consumed): `GET /dictionaries/dictionary/{id}` , `GET /dictionaries/batch?ids=...` , `GET /words/batch?ids=...` .

6.2 Requirements on the backend (Android blockers)

Requirement	Backend task	Blocks
Language list extended to the 19 codes of §8.1 (<code>supported.languages</code>)	one-line property + docs	Language expansion release — PT/NL dictionaries already fail sync today
Word meta documentation corrected to the §4 contract (current <code>Word.java</code> comment describes a different, fictional shape)	docs fix	Web/iOS/autofil implementing the wrong schema
Keycloak + JWT security live (BE Phases 2–3)	P2/P3	Login feature; release itself
User profile endpoints (<code>/users/...</code>)	P2-06	Account screen, profile
API Gateway as single origin	P5	Final base URL, HTTPS
Marketplace endpoints	P4-06	Forum tab implementation
Autofil suggestions	P6 (V2)	Suggestion feature (V2)

7. Current State — Honest Summary

Working offline product with a disciplined architecture; the gaps are exactly the integration-facing ones: **no auth** (guest UUID, `TOD0: getActiveUserUseCase` in sync), **LAN base URL** in both debug and release, **forum stub**, and a **language mismatch** with the backend validator (Android 10 vs backend 8 — pt, nl unaccepted).

8. Roadmap — Step by Step

Ordering principle: A1 is pure Android work and starts immediately; A2–A3 depend on backend Phases 2–5; A4 on Phase 4. Workstreams run in parallel with the backend and must not block each other. **Release gate: Android + Backend ready together.**

A1 — Language Expansion (immediate)

Add nine dictionary languages with **full grammar profiles** in `LanguageInputCard` — the same first-class treatment existing languages get, not minimal free-text stubs:

Polish (pl), Ukrainian (uk), Russian (ru), Greek (el), Arabic (ar), Farsi (fa), Japanese (ja), Chinese — Simplified only (zh), Korean (ko). Total after expansion: **19 languages**.

A1.1 New field keys

Seven new `FieldKey` entries. `reading` should be declared **first** in the enum so readings serialize/display before other forms (safe: existing languages never emit it); the rest are appended after `aux`:

metaKey	Field	Kind	Languages
<code>reading</code>	kana reading / pinyin	text	ja (all types), zh (all types)
<code>aspect</code>	aspect counterpart (perfective partner of an imperfective verb)	text	pl, uk, ru verbs
<code>root</code>	consonantal root	text	ar nouns & verbs
<code>stem</code>	present stem	text	fa verbs
<code>measure</code>	measure word / classifier	text	zh nouns
<code>class</code>	conjugation class	choice	ja verbs (<code>group1</code> / <code>group2</code> / <code>irregular</code>), ja adjectives (<code>i</code> / <code>na</code>)
<code>polite</code>	polite present (해요체)	text	ko verbs & adjectives

Implementation notes: `class` stores **stable ASCII codes** while the chips show localized labels — the current `ChoiceForm` renders the stored string directly (fine for `haben` / `sein`), so it needs a small labeled-choice variant. Every new `FieldKey` needs a `labelRes` string in the default `strings.xml` (translations into locale folders can follow; fallback is fine).

A1.2 Per-language grammar profiles and storage examples

Gender chips render as **articles** where the language has them (only Greek among the nine) and as localized grammatical labels otherwise. Arabic stores the bare citation form — the definite article `ال` is a bound prefix and is *not* composed into the surface.

Polish — genders m/f/n (labels); nouns: plural; verbs: present (3sg) + aspect; adjectives: comparative/superlative (morphological).

```
word: ["jabłko"]      meta: {"lang":"pl","type":"noun","genders":["n"],"fields":{"plural":["jabłka"]}}
word: ["kupować"]     meta: {"lang":"pl","type":"verb","fields":{"present":["kupuje"],"aspect":["kupić"]}}
```

Ukrainian — same profile as Polish.

```
word: ["книга"]      meta: {"lang":"uk","type":"noun","genders":["f"],"fields":{"plural":["книги"]}}
word: ["купувати"]   meta: {"lang":"uk","type":"verb","fields":{"present":["купує"],"aspect":["купити"]}}
```

Russian — as Polish; adjective comparison carries the irregular-only hint (regular -ee needs no entry, but хороший → лучше does).

```
word: ["яблоко"]      meta: {"lang":"ru","type":"noun","genders":["n"],"fields":{"plural":["яблоки"]}}
word: ["покупать"]     meta: {"lang":"ru","type":"verb","fields":{"present":["покупает"],"aspect":["купить"]}}
word: ["хороший"]      meta: {"lang":"ru","type":"adjective","fields":{"comparative":["лучше"],"superlative":["лучший"]}}
```

Greek — genders m/f/n with articles ο/η/το composed into the surface (extend [articlesByLanguage](#); [extractBaseWord](#) must strip them); nouns: plural; verbs: past (aorist); adjectives: comparison with irregular hint (periphrastic πιο is the regular default).

```
word: ["ο άνθρωπος"] meta: {"lang":"el","type":"noun","genders":["m"],"fields":{"plural":["άνθρωποι"]}}
word: ["γράφω"]       meta: {"lang":"el","type":"verb","fields":{"past":["έγραφα"]}}
```

Arabic — genders m/f (labels, no article composition); nouns: plural (broken plurals are the whole point) + root; verbs: present + root; adjectives: feminine (irregular hint — ة is the regular default). RTL.

```
word: ["كتاب"]        meta: {"lang":"ar","type":"noun","genders":["m"],"fields":{"plural":["كتب"],"root":["ك ت ب"]}}
word: ["كتب"]         meta: {"lang":"ar","type":"verb","fields":{"present":["يكتب"],"root":["ك ت ب"]}}
```

Farsi — no gender; nouns: plural (irregular hint — captures borrowed Arabic broken plurals like كتب → کتاب; regular -ها needs no entry); verbs: present stem (the irregular core of Farsi conjugation); adjectives: nothing (fully regular -ترین/-تر, same reasoning that excludes Turkish). RTL.

```
word: ["كتاب"]        meta: {"lang":"fa","type":"noun","fields":{"plural":["كتب"]}}
word: ["خریدن"]       meta: {"lang":"fa","type":"verb","fields":{"stem":["خر"]}}
```

Japanese — no gender; every word type: reading (kana); verbs: + class choice (group1 う / group2 る / irregular); adjectives: + class choice (i / na).

```
word: ["犬"]           meta: {"lang":"ja","type":"noun","fields":{"reading":["いぬ"]}}
word: ["食べる"]       meta: {"lang":"ja","type":"verb","fields":{"reading":["たべる"],"class":["group2"]}}
word: ["静か"]         meta: {"lang":"ja","type":"adjective","fields":{"reading":["しずか"],"class":["na"]}}
```

Chinese (Simplified) — no gender; every word type: reading (pinyin, with tone marks); nouns: + measure word. Simplified only; a traditional variant can be split out later if ever needed.

```
word: ["书"]      meta: {"lang":"zh","type":"noun","fields":{"reading":["shū"],"measure":["本"]}}
word: ["买"]      meta: {"lang":"zh","type":"verb","fields":{"reading":["mǎi"]}}
```

Korean — no gender, no reading (hangul is phonetic); verbs and adjectives (which conjugate identically): polite present 해요체 — it exposes the stem irregularities; nouns: base word only.

```
word: ["책"]      meta: {"lang":"ko","type":"noun"}
word: ["먹다"]     meta: {"lang":"ko","type":"verb","fields":{"polite":["먹어요"]}}
word: ["예쁘다"]  meta: {"lang":"ko","type":"adjective","fields":{"polite":["예뻐요"]}}
```

A1.3 Separators and RTL

Non-Latin scripts use their own separators — **no mixed-direction lines**. Display-layer only; storage is unaffected (surfaces are a JSON array, meanings never share a string).

Scripts	Alternative separator (between meanings)	Column separator (between forms)
Latin, Cyrillic (uk, ru), Greek (el), Korean	/	•
Arabic, Farsi (RTL)	, (U+060C)	؛ (U+061B)
Japanese	• (U+30FB)	、
Chinese	、	；

RTL checklist: display lines for ar/fa dictionaries must be composed RTL end-to-end; run a manual pass over `LanguageInputCard`, word list rows, and both practice screens with an Arabic dictionary before calling A1 done.

A1.4 Backend lockstep (hard dependency)

The backend language validator must ship the expanded list **before or with** this feature:

```
supported.languages=EN,DE,FR,ES,IT,PT,NL,TR,AZ,LT,PL,UK,AR,FA,JA,ZH,KO,EL,RU
```

Backend uppercases before validating, so Android's lowercase codes pass. Note pt and nl are missing from the backend list **today** — a PT/NL dictionary already fails upload and sits permanently unsynced. This property change should be a backend roadmap task.

A2 — Authentication (with backend Phases 2-3)

Implement the cross-client auth contract (Integration doc §6): Authorization Code + PKCE against Keycloak via **AppAuth / Custom Tabs**, tokens in encrypted storage, an OkHttp authenticator for refresh, `offline_access` scope. Replace `GUEST_USER_ID` with `getActiveUserUseCase`. On first login run the **guest-data migration**: rewrite local `fk_user_id` from the guest UUID to the real subject, flip all rows to `isSynced = false`, let the normal upload push them. At release the app is **online + login required**: the login wall comes before the main nav graph; the offline-first sync engine stays as the resilience layer underneath, not as an anonymous mode.

A3 — Environments & Networking (with backend Phase 5)

Replace the hardcoded LAN URL with per-build-type configuration (dev = local/LAN + cleartext permitted; staging/prod = HTTPS gateway origin only). Point at the API Gateway once BE P5 lands,

remove the cleartext exception outside dev, and adopt the batch endpoints in sync where they reduce round-trips.

A4 — Marketplace / Forum Tab (with backend Phase 4)

Replace the stub with browsing, language-pair filtering, popularity, and import against `/marketplace/...`. Import lands in the user's vault (backend event chain) and the imported dictionary then arrives through normal sync.

A5 — Release Readiness

Login-required launch flow, forced sync-on-login, RTL pass done, 19-language QA matrix, Play listing. Gate: backend Phases 2–5 live, both sides tested end-to-end.

Later (V2)

Autofil suggestions in the create-word screen (`GET /autofil?word=...&from=...&to=...`), practice statistics sync (the local `level` field is per-device until a backend design exists).

9. Related Documentation

- **Verborum Frontend-Backend Integration** — the cross-platform contract this document defers to for auth, environments, and API details (backend repo, `docs/integration/frontend-backend-integration.md`).
- **Backend agent docs** — `verborum_ms/docs/agent/` (`verborum.md` state & endpoints, `roadmap.md` phased plan with task IDs, `security.md` , `rabbitmq.md`).
- **This repo's `.claude/`** — Android-specific agents & skills for Claude Code development.